

- UNIDADE II – Programação Orientada a Aspectos (POA) com Spring Boot
 - Objetivos da Unidade
- 1. Introdução à Programação Orientada a Aspectos
 - Motivação
 - Exemplo sem POA
- 2. Interesses Transversais
 - Conceito
 - Exemplos
- 3. Problemas da Programação Orientada a Objetos
 - Espalhamento (Scattering)
 - Exemplo
 - Entrelaçamento (Tangling)
 - Exemplo
- 4. O que é Programação Orientada a Aspectos?
 - Estrutura Tradicional
 - Estrutura com POA
- 5. Conceitos Fundamentais
 - Aspect
 - Exemplo
 - Join Point
 - Pointcut
 - Exemplo
 - Advice
 - Como obter atributos do método no Advice (teoria + prática)
 - Exemplo didático
 - O que foi obtido no exemplo
 - Detalhes de Pointcut (como ler a expressão)
 - Designators importantes
 - Composição lógica
 - Onde encontrar mais detalhes (Pointcut)
 - Detalhes de Advice (como escolher o tipo)
 - Explicando os dois exemplos
 - Onde encontrar mais detalhes (Advice)
- 6. Spring AOP
 - O que é Spring AOP?
 - Dependência Maven
 - Como o Spring aplica o aspecto

- 7. Tipos de Advice
 - Before
 - After
 - AfterReturning
 - AfterThrowing
 - Around
- 8. Exemplo de Logging com Spring AOP
 - Serviço
 - Aspecto
 - Resultado
- 9. Aspectos Baseados em Anotações
 - Criando uma Anotação
 - Utilizando a Anotação
 - Aspecto de Auditoria
- 10. Estudo de Caso
 - Sistema de Pedidos
 - Requisitos
 - Classe de Negócio
 - Aspecto de Log
 - Aspecto de Monitoramento
- 11. Boas Práticas
 - Utilize Aspectos Para
 - Evite Aspectos Para
 - Desvantagens do uso de aspectos
- 12. Limitações do Spring AOP
 - Funciona com
 - Não funciona com
 - Self-invocation
- 13. Exemplo Prático no Projeto da Turma
 - O que foi adicionado
 - Código do aspecto
 - Como demonstrar em sala
 - Objetivo didático da demo
- 14. Roteiro para Apresentação em Sala (35–45 min)
- 15. Exercício Prático
 - Serviços
 - Aspectos
 - Requisitos

- [16. Questões para Fixação](#)
- [Resumo da Unidade](#)

UNIDADE II – Programação Orientada a Aspectos (POA) com Spring Boot

Carga Horária: 12 horas-aula **Foco:** material conceitual para apresentação em sala

Objetivos da Unidade

Ao final desta unidade, o aluno deverá ser capaz de:

- Identificar interesses transversais em aplicações orientadas a objetos.
- Compreender os problemas de espalhamento e entrelaçamento de código.
- Utilizar o Spring AOP para implementar aspectos.
- Aplicar aspectos para auditoria, logging, segurança e monitoramento.
- Avaliar quando utilizar Programação Orientada a Aspectos em aplicações Spring Boot.

1. Introdução à Programação Orientada a Aspectos

Motivação

A Programação Orientada a Objetos (POO) trouxe importantes avanços para a construção de sistemas por meio dos conceitos de:

- Encapsulamento
- Herança
- Polimorfismo
- Abstração

Entretanto, algumas funcionalidades acabam se espalhando por diversas classes da aplicação, dificultando a manutenção do código.

Exemplos:

- Registro de logs
- Auditoria
- Segurança
- Monitoramento
- Controle transacional
- Tratamento de exceções

Essas funcionalidades são conhecidas como **interesses transversais**.

Exemplo sem POA

```
@Service
public class ClienteService {

    public Cliente buscar(Long id) {

        System.out.println("Iniciando busca");

        Cliente cliente = repository.findById(id)
            .orElseThrow();

        System.out.println("Busca finalizada");

        return cliente;
    }
}
```

Observe que a lógica de log está misturada com a lógica de negócio.

2. Interesses Transversais

Conceito

Interesses transversais são funcionalidades que afetam múltiplas partes do sistema e que normalmente não pertencem diretamente à regra de negócio.

Exemplos

Interesse Transversal	Aplicação
Logging	Registro de operações
Segurança	Controle de acesso
Auditoria	Rastreabilidade
Monitoramento	Tempo de resposta
Transações	Commit e Rollback
Tratamento de Erros	Captura de exceções

3. Problemas da Programação Orientada a Objetos

Espalhamento (Scattering)

O mesmo código aparece em diversos locais.

Exemplo

```
logger.info("Iniciando operação");
```

Essa instrução pode aparecer em dezenas ou centenas de métodos.

Entrelaçamento (Tangling)

Ocorre quando diferentes responsabilidades são misturadas em um mesmo método.

Exemplo

```
public Pedido criarPedido() {  
  
    logger.info("Criando pedido");  
  
    validarPermissao();  
  
    Pedido pedido = processarPedido();  
  
    logger.info("Pedido criado");  
  
    return pedido;  
}
```

Nesse caso temos:

- Regra de negócio
- Segurança
- Logging

Tudo misturado em um único método.

4. O que é Programação Orientada a Aspectos?

A Programação Orientada a Aspectos (POA) é um paradigma que visa separar os interesses transversais da lógica principal do sistema.

Estrutura Tradicional

```
PedidoService  
├─ Regra de negócio  
├─ Logging  
├─ Segurança  
└─ Auditoria
```

Estrutura com POA

```
PedidoService
└─ Regra de negócio
```

```
LogAspect
SegurancaAspect
AuditoriaAspect
```

Resultado esperado:

- Código de negócio mais limpo
- Menos repetição
- Padronização de comportamento técnico

5. Conceitos Fundamentais

Aspect

Representa um interesse transversal. É uma classe que encapsula comportamento transversal.

Exemplo

```
@Aspect
@Component
public class LogAspect {
}
```

Join Point

Representa um ponto durante a execução da aplicação onde um aspecto pode atuar. No Spring AOP, tipicamente é a execução de um método de um bean Spring.

Exemplos:

- Execução de métodos
 - Retorno de métodos
 - Lançamento de exceções
-

Pointcut

Define quais Join Points serão interceptados. É a expressão que seleciona onde o aspecto vai atuar.

Exemplo

```
execution(* br.com.exemplo.service.*.*(..))
```

Significa:

- Qualquer método
- De qualquer classe
- Dentro do pacote service

Outro exemplo usando **within**:

```
within(br.unifor.produtosapi.service..)
```

Advice

Representa a ação executada quando um Pointcut é atingido.

Tipos principais:

- **@Before**: antes do método
- **@After**: depois do método (com sucesso ou erro)
- **@AfterReturning**: apenas quando termina com sucesso
- **@AfterThrowing**: apenas quando ocorre exceção

- `@Around`: envolve a execução (antes e depois); o mais poderoso
-

Como obter atributos do método no Advice (teoria + prática)

No Spring AOP, os detalhes do método interceptado chegam pelo objeto de join point:

- `JoinPoint` para `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`
- `ProceedingJoinPoint` para `@Around`

Com ele, é possível acessar assinatura e metadados do método.

Exemplo didático

```
@Around("execution(* br.unifor.produtosapi.service..*(..))")
public Object inspecionarMetodo(ProceedingJoinPoint pjp) throws Throwable {
    MethodSignature signature = (MethodSignature) pjp.getSignature();
    Method metodo = signature.getMethod();

    String nomeMetodo = metodo.getName();
    Class<?> classeRetorno = metodo.getReturnType();
    Class<?>[] tiposParametros = metodo.getParameterTypes();
    Annotation[] anotacoes = metodo.getDeclaredAnnotations();
    Object[] valoresArgumentos = pjp.getArgs();
    String classeAlvo = pjp.getTarget().getClass().getSimpleName();

    Object retorno = pjp.proceed();
    return retorno;
}
```

O que foi obtido no exemplo

- Nome do método (`metodo.getName()`)
 - Tipo de retorno (`metodo.getReturnType()`)
 - Tipos dos parâmetros (`metodo.getParameterTypes()`)
 - Anotações do método (`metodo.getDeclaredAnnotations()`)
 - Valores dos argumentos na chamada (`pjp.getArgs()`)
 - Classe real do bean alvo (`pjp.getTarget()`)
-

Detalhes de Pointcut (como ler a expressão)

Formato mais comum:

```
execution(modificadores? retorno pacote.Classe.metodo(parametros) excecoes?)
```

Exemplo:

```
execution(* br.unifor.produtosapi.service..*(..))
```

Leitura:

- `*` (retorno): qualquer tipo de retorno
- `br.unifor.produtosapi.service..`: qualquer classe nesse pacote e subpacotes
- `*`: qualquer nome de método
- `(..)`: qualquer quantidade e tipo de parâmetros

Designators importantes

- `execution(...)`: filtra por assinatura do método
- `within(...)`: filtra por tipo/classe
- `args(...)`: filtra pelos tipos dos argumentos em runtime
- `@annotation(...)`: filtra métodos anotados
- `this(...)` e `target(...)`: filtra tipo do proxy/alvo

Composição lógica

- `&&` (E)
- `||` (OU)
- `!` (NÃO)

Exemplo composto:

```
@Pointcut("within(br.unifor.produtosapi.service..*) && !execution(* *.*Test.*(..))")
public void servicosSemTeste() {}
```

Onde encontrar mais detalhes (Pointcut)

- Spring Framework Reference (AOP): <https://docs.spring.io/spring-framework/reference/core/aop.html>
- Spring @AspectJ Pointcuts: <https://docs.spring.io/spring-framework/reference/core/aop/aspectj/pointcuts.html>
- AspectJ Pointcut Semantics: <https://eclipse.dev/aspectj/doc/released/progguide/semantics-pointcuts.html>

Detalhes de Advice (como escolher o tipo)

- **@Before**: validações, trilha de início, auditoria de entrada
- **@After**: limpeza final (sempre executa, com sucesso ou erro)
- **@AfterReturning**: usa o valor de retorno (**returning** = "resultado")
- **@AfterThrowing**: trata/loga exceção (**throwing** = "ex")
- **@Around**: mede tempo, altera fluxo, encapsula chamada (**proceed()**)

Exemplos de binding em Advice:

```
@AfterReturning(pointcut = "execution(* ..service..*(..))", returning = "resultado")
public void aposSucesso(Object resultado) { }
```

```
@AfterThrowing(pointcut = "execution(* ..service..*(..))", throwing = "ex")
public void aposErro(Exception ex) { }
```

Explicando os dois exemplos

@AfterReturning:

- Executa **somente quando o método termina com sucesso** (sem lançar exceção).
- O atributo `pointcut` define **quais métodos** serão monitorados.
- O atributo `returning = "resultado"` diz ao Spring para pegar o valor retornado pelo método interceptado e injetá-lo no parâmetro `resultado` do advice.
- Uso comum: auditoria de sucesso, métricas de resposta e logging do retorno.

Exemplo prático de leitura: se `produtoService.buscarPorId(10)` retornar um `Produto`, o objeto retornado chega em `resultado`.

`@AfterThrowing`:

- Executa **somente quando o método lança exceção**.
- O atributo `pointcut` também define o conjunto de métodos interceptados.
- O atributo `throwing = "ex"` vincula a exceção lançada ao parâmetro `ex` do advice.
- Uso comum: logging de erro padronizado, auditoria de falhas e enriquecimento de observabilidade.

Exemplo prático de leitura: se `produtoService.excluir(id)` lançar `RegraNegocioException`, essa exceção chega em `ex`, permitindo registrar `ex.getClass().getSimpleName()` e `ex.getMessage()`.

Resumo rápido:

- `@AfterReturning` trata o **caminho de sucesso**.
- `@AfterThrowing` trata o **caminho de erro**.

Onde encontrar mais detalhes (Advice)

- Spring @AspectJ Advice: <https://docs.spring.io/spring-framework/reference/core/aop/aspectj/advice.html>
 - API AspectJ de anotações: <https://eclipse.dev/aspectj/doc/released/aspectj5rt-api/org/aspectj/lang/annotation/package-summary.html>
 - API de JoinPoint e ProceedingJoinPoint: <https://eclipse.dev/aspectj/doc/released/runtime-api/org/aspectj/lang/JoinPoint.html>
-

6. Spring AOP

O que é Spring AOP?

Spring AOP é o módulo de Programação Orientada a Aspectos fornecido pelo Spring Framework.

Ele permite implementar aspectos de forma simples utilizando anotações, sem precisar de configuração complexa.

Dependência Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

Como o Spring aplica o aspecto

- O Spring cria proxies para beans gerenciados (`@Service`, `@Component`, `@Repository`, `@RestController`).
 - Ao chamar o bean pelo container, o proxy executa o advice antes/depois do método alvo.
-

7. Tipos de Advice

Before

Executa antes do método.

```
@Before(  
    "execution(* br.com.exemplo.service.*.*(..))")  
public void logAntes() {  
  
    System.out.println("Método iniciado");  
}
```

After

Executa após o término do método.

```
@After(  
    "execution(* br.com.exemplo.service.*.*(..))")  
public void logDepois() {  
  
    System.out.println("Método finalizado");  
}
```

AfterReturning

Executa apenas quando não ocorre erro.

```
@AfterReturning(  
    pointcut =  
    "execution(* br.com.exemplo.service.*.*(..))",  
    returning = "resultado")  
public void sucesso(Object resultado) {  
  
    System.out.println(resultado);  
}
```

AfterThrowing

Executa quando uma exceção é lançada.

```
@AfterThrowing(  
    pointcut =  
        "execution(* br.com.exemplo.service.*.*(..))",  
    throwing = "ex")  
public void erro(Exception ex) {  
  
    System.out.println(ex.getMessage());  
}
```

Around

Executa antes e depois do método.

É o tipo de Advice mais poderoso e mais usado em observabilidade.

```
@Around(  
    "execution(* br.com.exemplo.service.*.*(..))")  
public Object monitorar(  
    ProceedingJoinPoint joinPoint)  
    throws Throwable {  
  
    long inicio = System.currentTimeMillis();  
  
    Object retorno = joinPoint.proceed();  
  
    long fim = System.currentTimeMillis();  
  
    System.out.println(  
        "Tempo: " + (fim - inicio));  
  
    return retorno;  
}
```

8. Exemplo de Logging com Spring AOP

Serviço

```
@Service  
public class ProdutoService {
```

```
public void cadastrar() {  
    System.out.println("Produto cadastrado");  
}
```

Aspecto

```
@Aspect  
@Component  
public class LogAspect {  
    @Before(  
        "execution(* br.com.exemplo.service.*.*(..))"  
    )  
    public void registrarLog() {  
        System.out.println(  
            "Executando método...");  
    }  
}
```

Resultado

```
Executando método...  
Produto cadastrado
```

9. Aspectos Baseados em Anotações

Criando uma Anotação

```
@Target(ElementType.METHOD)  
@Retention(RetentionPolicy.RUNTIME)
```



```
public @interface Auditavel {  
}
```

Utilizando a Anotação

```
@Service  
public class ClienteService {  
  
    @Auditavel  
    public void excluir(Long id) {  
  
        System.out.println("Cliente excluído");  
    }  
}
```

Aspecto de Auditoria

```
@Aspect  
@Component  
public class AuditoriaAspect {  
  
    @Before("@annotation(Auditavel)")  
    public void auditar() {  
  
        System.out.println(  
            "Operação auditada");  
    }  
}
```

10. Estudo de Caso

Sistema de Pedidos

Requisitos

- Registrar logs.
 - Auditar operações.
 - Medir tempo de execução.
-

Classe de Negócio

```
@Service
public class PedidoService {

    public void gerarPedido() {

        System.out.println(
            "Pedido gerado");
    }
}
```

Aspecto de Log

```
@Aspect
@Component
public class LogAspect {

    @Before(
        "execution(* *..PedidoService.*(..))")
    public void log() {

        System.out.println("Início");
    }
}
```

Aspecto de Monitoramento

```
@Aspect
@Component
public class MonitoramentoAspect {
```

```
@Around(  
    "execution(* *..PedidoService.*(..))")  
public Object medirTempo(  
    ProceedingJoinPoint pjp)  
    throws Throwable {  
  
    long inicio = System.currentTimeMillis();  
  
    Object retorno = pjp.proceed();  
  
    long fim = System.currentTimeMillis();  
  
    System.out.println(  
        "Tempo: " + (fim - inicio));  
  
    return retorno;  
    }  
}
```

11. Boas Práticas

Utilize Aspectos Para

- Logging
- Auditoria
- Segurança
- Monitoramento
- Métricas
- Transações

Evite Aspectos Para

- Regras de negócio
- Cálculos de domínio
- Validações de domínio
- Processamento principal da aplicação

Regra prática:

- Se for transversal e repetitivo, AOP ajuda.
- Se for regra do domínio, mantenha no service/domain.

Desvantagens do uso de aspectos

Apesar dos ganhos de organização, AOP também traz custos e riscos:

- **Maior complexidade de entendimento:** o comportamento final pode estar distribuído entre service e aspectos, exigindo leitura de múltiplos pontos.
- **Fluxo de execução menos explícito:** para quem lê apenas o método de negócio, parte da lógica técnica fica "invisível" (executada por advice).
- **Debug mais trabalhoso:** em alguns cenários, o stack trace inclui proxies/interceptadores e dificulta rastrear causa raiz rapidamente.
- **Risco de pointcut muito amplo:** expressões genéricas podem interceptar métodos demais e gerar efeitos colaterais.
- **Overhead de execução:** cada interceptação adiciona custo (proxy + advice), principalmente perceptível em trechos de alta frequência.
- **Dependência de convenções da equipe:** sem padrão claro, diferentes aspectos podem conflitar em ordem, granularidade e responsabilidade.
- **Limitações do Spring AOP:** por ser proxy-based, não cobre todos os cenários (ex.: self-invocation, construtores, métodos privados).

Como mitigar:

- Definir pointcuts específicos e bem nomeados.
- Evitar lógica pesada dentro do advice.
- Documentar a ordem e o propósito de cada aspecto.
- Monitorar desempenho em produção com métricas.

12. Limitações do Spring AOP

O Spring AOP trabalha apenas com Beans gerenciados pelo Spring.

Funciona com

```
@Service
@Repository
@Component
@RestController
```

Não funciona com

```
new Cliente();
```

Também não intercepta:

- Métodos privados
- Construtores
- Campos/Atributos

Self-invocation

Chamada de um método para outro dentro da mesma classe normalmente não passa pelo proxy, então o advice pode não disparar.

```
@Service
public class PedidoService {
    public void processar() {
        validar(); // NÃO passa pelo proxy – advice não dispara
    }
    public void validar() { ... }
}
```

Para cenários que exigem interceptar construtores ou self-invocations, utiliza-se AspectJ completo.

13. Exemplo Prático no Projeto da Turma

O que foi adicionado

1. Dependências no Maven:

- arquivo: `backend/produtos-api/pom.xml`
- `org.springframework.boot:spring-boot-starter-aspectj`
- `org.springdoc:springdoc-openapi-starter-webmvc-ui:3.0.0` (compatível com Spring Boot 4 / Spring 7)

2. Aspecto de observabilidade:

- arquivo: `backend/produtos-api/src/main/java/br/unifor/produtosapi/aspect/ObservabilidadeAspect.java`
- intercepta métodos de `controller` e `service`
- registra entrada de parâmetros, saída de retorno e tempo/status de execução

3. Persistência de log em arquivo:

- arquivo: `backend/produtos-api/src/main/resources/application.properties`
- `logging.file.name=logs/produtos-api.log`
- rotação configurada (`max-file-size`, `max-history`, `total-size-cap`)

Código do aspecto

```
@Aspect
@Component
public class ObservabilidadeAspect {

    @Pointcut("within(br.unifor.produtosapi.controller..*) ||
within(br.unifor.produtosapi.service..*)")
    public void camadasApi() {}

    @Before("camadasApi()")
    public void logarEntradas(JoinPoint joinPoint) {
        // registra todos os parametros recebidos
    }

    @AfterReturning(pointcut = "camadasApi()", returning = "retorno")
    public void logarRetornos(JoinPoint joinPoint, Object retorno) {
        // registra o retorno no caminho de sucesso
    }
}
```

```
}

@Around("camadasApi()")
public Object registrarTempoEStatus(ProceedingJoinPoint joinPoint) throws
Throwable {
    // mede tempo total e status (sucesso/erro)
    // executa método alvo
    // log de fim com tempo e status
}
}
```

Como demonstrar em sala

1. Subir backend:

```
cd backend/produtos-api
./mvnw spring-boot:run
```

2. Chamar endpoint:

```
curl http://localhost:8080/tipos
```

3. Mostrar logs no console e no arquivo:

- ENTRADA metodo=... parametros=[...]
- SAIDA metodo=... retorno=...
- FIM metodo=... tempoMs=... sucesso=true
- arquivo: backend/produtos-api/logs/produtos-api.log

4. Forçar erro (id inexistente em update/delete) para mostrar:

- sucesso=false
- tipo da exceção e mensagem

Objetivo didático da demo

- Provar que o service/controller não precisou receber código de log.
- Mostrar separação de responsabilidades na prática.

- Evidenciar ganho de padronização e rastreabilidade.
-

14. Roteiro para Apresentação em Sala (35–45 min)

1. Problema real: código repetido (5 min)
 2. Conceitos: Aspect, Join Point, Pointcut, Advice (10 min)
 3. Spring AOP e proxies (8 min)
 4. Limitações e armadilhas, incluindo self-invocation (7 min)
 5. Exemplo no projeto da turma — demo ao vivo (10–15 min)
-

15. Exercício Prático

Desenvolva uma API Spring Boot contendo:

Serviços

- UsuarioService
- ProdutoService
- PedidoService

Aspectos

- LogAspect
- AuditoriaAspect
- TempoExecucaoAspect

Requisitos

1. Registrar entrada em todos os métodos.
2. Registrar saída dos métodos.

3. Medir tempo de execução.
 4. Auditar métodos anotados com `@Auditavel`.
 5. Não incluir código de log dentro dos serviços.
-

16. Questões para Fixação

1. O que são interesses transversais? O que os caracteriza?
 2. Explique com exemplos a diferença entre espalhamento (scattering) e entrelaçamento (tangling).
 3. O que é um Aspect?
 4. O que é um Join Point?
 5. O que é um Pointcut?
 6. O que é um Advice? Quais são os tipos existentes?
 7. Qual a diferença entre `@Before` e `@Around`?
 8. Por que `@Around` é tão usado em observabilidade?
 9. O que é self-invocation e por que pode impedir a interceptação?
 10. Quando utilizar Spring AOP? Quando evitar?
 11. Quais são as limitações do Spring AOP?
-

Resumo da Unidade

Nesta unidade foram estudados:

- Motivação para Programação Orientada a Aspectos.
- Interesses transversais.
- Problemas de espalhamento (scattering) e entrelaçamento (tangling).
- Conceitos fundamentais da POA: Aspect, Join Point, Pointcut, Advice.
- Spring AOP e o modelo baseado em proxies.
- Tipos de Advice: Before, After, AfterReturning, AfterThrowing, Around.
- Aspectos baseados em anotações.
- Implementação de logging, auditoria e monitoramento.
- Boas práticas e limitações do Spring AOP.
- Exemplo prático aplicado ao projeto da turma.

A Programação Orientada a Aspectos é uma importante técnica para melhorar a modularização, manutenção e reutilização de código em aplicações corporativas

desenvolvidas com Spring Boot.